

Kubernetes and Cloud Native Associate - KCNA

Kubernetes Resources

Deployments Rolling Updates and Rollbacks

Welcome to this comprehensive guide on managing deployments in Kubernetes. In this article, you'll learn how to perform rolling updates, roll back changes, and effectively manage deployment revisions.

When you create a deployment, Kubernetes automatically triggers a rollout, establishing your initial deployment revision (revision one). Later, when you upgrade your application (for example, by updating the container image version), a new rollout is triggered and a new deployment revision (revision two) is created.

Rollout and Versioning



Revision 1



nginx:1.7.0



nginx:1.7.0



nginx:1.7.0



nginx:1.7.0



nginx:1.7.0



nginx:1.7.0



nginx:1.7.0



nginx:1.7.0



nginx:1.7.0

Revision 2



nginx:1.7.1



nginx:1.7.1



nginx:1.7.1



nginx:1.7.1



nginx:1.7.1



nginx:1.7.1



nginx:1.7.1



nginx:1.7.1



nginx:1.7.1

This revision history lets you track changes and enables you to roll back to a previous version if necessary.

You can monitor the status of a rollout by running:

```
kubectl rollout status deployment/myapp-deployment
```

The output might be similar to:

```
Waiting for rollout to finish: 0 of 10 updated replicas are available...  
Waiting for rollout to finish: 1 of 10 updated replicas are available...  
...  
Waiting for rollout to finish: 9 of 10 updated replicas are available...  
deployment "myapp-deployment" successfully rolled out
```

To view the deployment's revision history, use:

```
kubectl rollout history deployment/myapp-deployment
```

Deployment Strategies

There are two primary deployment strategies to consider when updating your application:

1. Recreate Strategy:

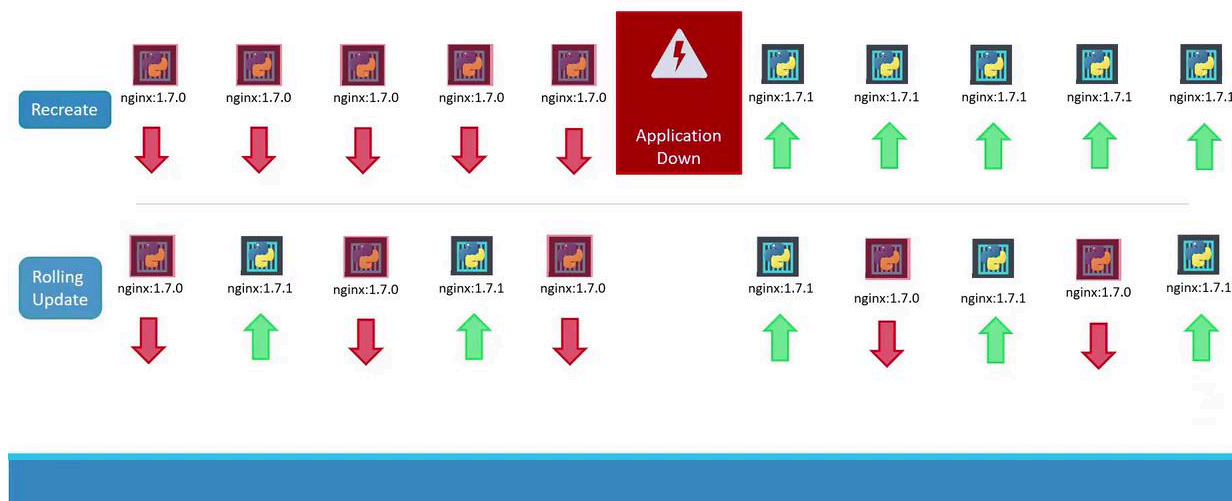
In this approach, all existing replicas are terminated before new ones are created. Although this creates a completely fresh environment, it results in downtime due to the gap between terminating the old pods and starting the new ones.

2. Rolling Update Strategy:

This strategy updates instances incrementally. Kubernetes gradually terminates old pods while simultaneously starting new ones, ensuring continuous

application availability. Rolling updates are applied by default unless a different strategy is specified during deployment creation.

Deployment Strategy



Tip

Kubernetes defaults to the rolling update strategy to minimize downtime during deployments.

Updating a Deployment

There are several ways to update a deployment, such as changing your container image, modifying labels, or adjusting the replica count. If you maintain a deployment configuration file, you can modify it directly. For example:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
```

```
labels:
  app: myapp
  type: front-end
spec:
  replicas: 3
  selector:
    matchLabels:
      type: front-end
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
      - name: nginx-container
        image: nginx:1.7.1
```

After updating the file, apply the changes using:

```
kubectl apply -f deployment-definition.yml
```

This command initiates a new rollout and creates a new deployment revision.

Alternatively, update the container image directly with:

```
kubectl set image deployment/myapp-deployment nginx-container=nginx:1.9.1
```



Caution

The `kubectl set image` command edits the live deployment configuration. This modification may not be reflected in your deployment definition file, so use it with care if you plan to maintain consistency through file-based updates.

Examining Deployment Details

To inspect detailed information about a deployment—including its strategy—run:

```
kubectl describe deployment myapp-deployment
```

When using the recreate strategy, you will see that the old replica set is scaled down to zero before the new replica set is scaled up. For example:

```
Namespace:          default
CreationTimestamp:   Sat, 03 Mar 2018 17:01:55 +0800
Labels:              app=myapp
                    type=front-end
Annotations:         deployment.kubernetes.io/revision: 2
                    kubernetes.io/change-cause: kubectl apply --filename=dep
Replicas:            5 updated | 5 total | 5 available | 0 unavailable
StrategyType:        Recreate
MinReadySeconds:     0
Containers:
  nginx-container:
    Image:            nginx:1.7.1
    Port:             <none>
Conditions:
  Type      Status  Reason
  Available  True    MinimumReplicasAvailable
  Progressing True    NewReplicaSetAvailable
OldReplicaSets: <none>
```

```
NewReplicaSet:      myapp-deployment-5c47d6ccc (5/5 replicas created)
Events:
  Type    Reason             Age   From                    Message
  ----    -
  Normal  ScalingReplicaSet  11m   deployment-controller   Scaled up replica set
  Normal  ScalingReplicaSet  1m    deployment-controller   Scaled down replica se
```

For rolling updates, the output shows a gradual transition:

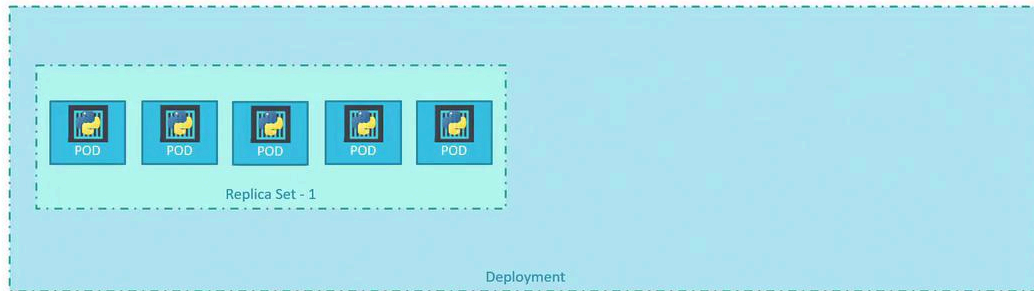
```
Namespace:          default
CreationTimestamp:   Sat, 03 Mar 2018 17:16:53 +0800
Labels:              app=myapp
                    type=front-end
Annotations:         deployment.kubernetes.io/revision: 2
                    kubernetes.io/change-cause: kubectl apply --filename=dep
Replicas:            5 desired | 1 updated | 4 available | 2 unavailable
StrategyType:        RollingUpdate
Pod Template:
  Labels:             app=myapp
                    type=front-end
Containers:
  nginx-container:
    Image:             nginx:1.7.1
    Port:              <none>
Conditions:
  Type            Status  Reason
  ----            -
  Available       True    MinimumReplicasAvailable
  Progressing     True    ReplicaSetUpdated
OldReplicaSets:   myapp-deployment-676749c58c (1/1 replicas created)
NewReplicaSet:    myapp-deployment-7d57dbbd8d (5/5 replicas created)
Events:
  Type    Reason             Age   From                    Message
  ----    -
  Normal  ScalingReplicaSet  11m   deployment-controller   Scaled up replica set
  Normal  ScalingReplicaSet  15s   deployment-controller   Scaled down replica se
  Normal  ScalingReplicaSet  56s   deployment-controller   Scaled down replica se
```

These examples clearly illustrate the differences between the recreate and rolling update strategies.

How Upgrades Work Under the Hood

When a deployment is initially created (for example, deploying five replicas), Kubernetes automatically creates a replica set that spawns the required pods. During an upgrade, a new replica set is generated for the updated containers while the pods from the previous replica set are gradually terminated according to the rolling update strategy.

Upgrades



You can verify the changes by listing the replica sets:

```
kubectl get replica sets
```

Before the upgrade, the old replica set displays the active pods, and after updating, you will see the new replica set with updated pod counts.

Rolling Back an Update

If you encounter issues with a new release, Kubernetes allows you to roll back to a previous deployment revision. To perform a rollback, simply execute:

```
kubectl rollout undo deployment/myapp-deployment
```

The output will confirm the rollback:

```
deployment "myapp-deployment" rolled back
```

Before executing the rollback, the new replica set might show all pods (for instance, five replicas) while the old replica set shows none. After the rollback, these counts are reversed.

Summary of Essential Commands

Below is a summary table of commands that are crucial for managing your deployments:

Command	Description
<code>kubectl create -f deployment-definition.yml</code>	Create a new deployment using the configuration file
<code>kubectl get deployments</code>	List all deployments in your cluster
<code>kubectl apply -f deployment-definition.yml</code>	Update an existing deployment with changes from the configuration file
<code>kubectl set image deployment/myapp-deployment nginx-container=nginx:1.9.1</code>	Update the container image in a running deployment
<code>kubectl rollout status deployment/myapp-deployment</code>	Monitor the status of the deployment rollout
<code>kubectl rollout undo deployment/myapp-deployment</code>	Roll back to the previous deployment revision

These commands empower you to create, update, monitor, and roll back deployments efficiently, ensuring minimal downtime and a smooth upgrade process.

For further insights, consider reviewing the [Kubernetes Documentation](#).



Watch Video

Watch video content

Previous

◀ Demo Deployments

Next

Demo Deployments Rolling Updates and Rollbacks ▶